ARCHITECTURE & SYSTEM SPECIFICATION

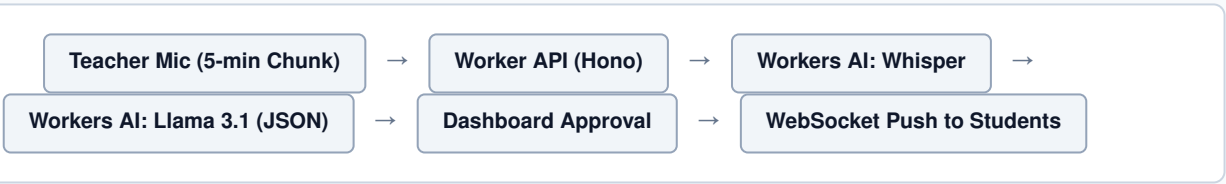
LiveLectureQuiz System Specification

Ephemeral Real-Time AI-Driven Classroom Assessment Platform Built on Cloudflare Edge Infrastructure

TARGET ARCHITECTURE	PRIMARY STACK	DEPLOYMENT MODE
Cloudflare Workers / Workers AI / KV	TypeScript + Hono + Native WebSockets	Serverless Zero-Cost Edge Tier

1. System Architecture & Top-Level Design

LiveLectureQuiz is an ultra-lightweight, real-time classroom feedback application designed to run with zero fixed infrastructure costs. The system captures live audio from a teacher's lecture in 5-minute chunks, transcribes it at the edge using Cloudflare Workers AI (Whisper), processes the text to generate multiple-choice attention-check questions (Llama 3.1 / Mistral), and streams approved questions to student devices instantly over WebSockets.



Component	Cloudflare Binding	Implementation Role	Free Allocation Limits
Backend API	Cloudflare Workers	API routing (Hono framework), WebSocket server, session management.	100,000 requests / day
Speech-to-Text	`env.AI` (Whisper)	Executes `@cf/openai/whisper-large-v3-turbo` on 5-min audio slices.	10,000 Neurons / day (~500 mins audio)
LLM Quiz Engine	`env.AI` (Llama / Mistral)	Executes `@cf/meta/llama-3.1-8b-instruct` using forced JSON mode.	Shared Neuron Pool (~150 queries/day)
Session State	`env.LECTURE_KV`	Stores active rooms, pending/approved question objects, student answers.	100,000 reads / 1,000 writes / day
Real-Time Delivery	Worker WebSockets	Direct bidirectional messaging to teacher dashboard and student clients.	Included in Workers execution

2. Data Models & State Persistence (Workers KV Schema)

Because data is stored in Workers KV, keys use structured namespaces for deterministic lookups. TTLs (Time-To-Live) of 86,400 seconds (24 hours) are applied to auto-purge old classroom data.

Key 1: Session Metadata — `session:{roomId}`

```
{
  "roomId": "room_101",
  "createdAt": 1782390123,
  "currentQuestionId": "q_9821a",
  "status": "active"
}
```

Key 2: Question Pool — `questions:{roomId}:{questionId}`

```
{
  "id": "q_9821a",
  "text": "What is the primary function of ATP in cell metabolism?",
  "options": [
    "Storing genetic code",
    "Providing immediate cellular energy",
    "Synthesizing lipid membranes",
    "Facilitating oxygen diffusion"
  ],
  "correctIndex": 1,
  "explanation": "ATP acts as the primary energy currency of the cell.",
  "approved": false,
  "timestamp": 1782390400
}
```

Key 3: Student Submissions — `responses:{roomId}:{questionId}`

```
{
  "responses": [
    { "studentId": "anon_82", "chosenIndex": 1, "timestamp": 1782390420 },
    { "studentId": "anon_14", "chosenIndex": 2, "timestamp": 1782390422 }
  ]
}
```

3. API Endpoints & Real-Time Protocol

3.1 Audio Ingestion & Quiz Generation Endpoint

Endpoint: `POST /api/room/:roomId/audio` **POST**

Payload: Binary `ArrayBuffer` or `multipart/form-data` containing `.webm`/`.mp3` audio chunk.

Execution Flow:

1. Receive binary audio buffer in Hono request handler.
2. Invoke Whisper AI: `await env.AI.run("@cf/openai/whisper-large-v3-turbo", { audio: [...new Uint8Array(buffer)] })`.
3. Extract plain text transcript from Whisper output.
4. Construct deterministic LLM prompt demanding 2 multiple-choice questions in strict JSON format.
5. Invoke Llama 3.1 AI using `response_format: { type: "json_object" }`.

6. Save generated question object to Workers KV under `questions:{roomId}:{questionId}` with `approved: false`.
7. Broadcast `NEW_QUESTION_PENDING` event to Teacher WebSocket.

3.2 WebSocket Communication Matrix

Route: `/ws/room/:roomId?role=[teacher|student]` WebSocket

Event Name	Source	Payload Schema	Target Broadcast & Action
`NEW_QUESTION_PENDING`	Server	`{ event, questionObject }`	**Teacher Only:** Renders question in review queue.
`APPROVE_QUESTION`	Teacher	`{ event: "APPROVE_QUESTION", questionId }`	**All Students:** Sets KV `approved: true` & displays quiz UI on phones.
`SUBMIT_ANSWER`	Student	`{ event: "SUBMIT_ANSWER", questionId, choiceIndex }`	**Teacher Only:** Appends choice to KV & updates live bar charts.
`CLOSE_QUESTION`	Teacher	`{ event: "CLOSE_QUESTION", questionId }`	**All Students:** Clears quiz screen; returns to waiting state.

4. Prompt Engineering & JSON Schema Binding

To ensure 100% reliable JSON parsing without runtime errors, system prompts are strictly structured with JSON schema constraints.

```
const SYSTEM_PROMPT = `You are an expert pedagogical assistant. Analyze the provided transcript chunk from a high
CRITICAL: You MUST respond ONLY with a valid JSON object matching this structure:
{
  "questions": [
    {
      "text": "Question string",
      "options": ["Option A", "Option B", "Option C", "Option D"],
      "correctIndex": 0,
      "explanation": "Brief explanation"
    }
  ]
}`;
```

5. Frontend Client Implementation Specification

Teacher Dashboard Requirements

- **Audio Chunking Engine:** Uses native `navigator.mediaDevices.getUserMedia` and `MediaRecorder` with `webm` mimeType. Slices and sends audio data via POST fetch every 300,000ms (5 mins).
- **Review Queue UI:** Displays incoming AI questions with "Approve & Push" and "Discard" buttons.
- **Live Histogram:** Dynamic HTML/CSS bar graph updating in real time as `SUBMIT_ANSWER` events arrive via WebSocket.

Student Client Requirements

- **Zero-Authentication Access:** Accessible via room code URL (e.g., `https://app.com/?room=101`). Generates anonymous persistent `studentId` in `localStorage`.
- **State-Driven Interface:** Switches seamlessly between "Listening to Lecture..." idle state and active 4-option radio quiz input upon receiving broadcast. Inputs disable immediately after submission to prevent double-voting.

6. Configuration for Agentic IDE (Google Antigravity)

> **Instructions for Antigravity Execution**
>
> Provide the prompt below directly to Antigravity to generate the complete, self-contained, and deployable codebase.

```
Please implement the LiveLectureQuiz system based on the following complete file structure:

1. `wrangler.jsonc`:
   - Configures `kv_namespaces` with binding `LECTURE_KV`.
   - Enables `ai` binding named `AI`.
   - Defines compatibility_date and main entry point `src/index.ts`.

2. `src/index.ts`:
   - Hono application setting up routes:
     - `POST /api/room/:roomId/audio` (Whisper transcription + LLM quiz generation).
     - `GET /ws/room/:roomId` (WebSocket endpoint handling Teacher/Student messaging).
     - Static file server serving `public/index.html`.

3. `public/index.html`:
   - Single-file monolithic frontend using Tailwind CSS CDN.
   - Contains both Teacher and Student UI logic toggled via URL param `?role=teacher`.
   - Includes MediaRecorder 5-minute slice script for teacher microphone stream.
   - Includes WebSocket auto-reconnect loop and live canvas/CSS bar chart renderer.

Ensure all TypeScript code compiles cleanly without external unmentioned dependencies beyond `hono` and `@cloudfl
```